

Apostila Java — Wrappers, Escopo, Parâmetros, Varargs, Classes Aninadas e Annotations

Jader Silva · [GitHub](#) · [LinkedIn](#)

Sumário

1. Wrappers — aprofundamento
 2. Importação Estática (import static)
 3. Escopo de Variáveis
 4. Passagem de Parâmetros: Valor x Referência
 5. Varargs
 6. Classes Aninadas: Internas, Locais e Anônimas
 7. Annotations
-

1. Wrappers — aprofundamento

1.1 Revisão rápida

Wrappers são classes que encapsulam tipos primitivos como objetos: `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character`, `Boolean`. Você já viu que, a partir do Java 9, os construtores (`new Integer(1)`) estão deprecated/removidos — o caminho correto é `Integer.valueOf(1)` ou simplesmente o autoboxing (`Integer x = 1;`).

1.2 Autoboxing e Unboxing

Autoboxing: conversão automática de primitivo para wrapper. **Unboxing**: conversão automática de wrapper para primitivo. O compilador faz isso implicitamente sempre que o contexto exige.

```
Integer obj = 10;      // autoboxing: int -> Integer
int prim = obj;       // unboxing: Integer -> int

List<Integer> lista = new ArrayList<>();
lista.add(5);         // autoboxing (int -> Integer)
int valor = lista.get(0); // unboxing (Integer -> int)
```

1.3 O cache de Integer/Short/Byte/Long (-128 a 127)

Para economizar memória, a JVM mantém um cache de objetos wrapper para valores inteiros entre -128 e 127. Isso afeta comparações com `==`:

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // true (cache)

Integer c = 200;
Integer d = 200;
System.out.println(c == d); // false (fora do cache, objetos novos)

// Regra de ouro: NUNCA compare wrappers com == , sempre use .equals()
System.out.println(c.equals(d)); // true
```

Atenção: isso não se aplica a `Float` e `Double` — eles nunca são cacheados, porque valores de ponto flutuante têm precisão/representação que tornam o cache pouco útil.

1.4 Métodos utilitários mais usados

Método	Exemplo	Retorno
<code>parseInt</code> / <code>parseShort</code> / <code>parseDouble</code> ...	<code>Integer.parseInt("42")</code>	primitivo (int)
<code>valueOf</code>	<code>Integer.valueOf("42")</code>	wrapper (Integer)
<code>toString</code>	<code>Integer.toString(42)</code>	String
<code>compare</code>	<code>Integer.compare(3, 7)</code>	int (negativo/0/positivo)
<code>MAX_VALUE</code> / <code>MIN_VALUE</code>	<code>Integer.MAX_VALUE</code>	constante do tipo
<code>isNaN</code> (Double/Float)	<code>Double.isNaN(x)</code>	boolean

Ponte com Python: `parseInt` / `parseDouble` são o equivalente Java ao `int(x)` / `float(x)` do Python quando você converte uma string vinda de `input()` ou de um arquivo.

2. Importação Estática (import static)

2.1 O que é

O `import static` permite usar membros **estáticos** (atributos e métodos) de uma classe sem precisar qualificá-los com o nome da classe. Você já usa isso sem perceber quando escreve testes com JUnit (`assertEquals` em vez de `Assertions.assertEquals`).

2.2 Sintaxe

```
// Sem import static
double raiz = Math.sqrt(25);
double pi = Math.PI;

// Com import static
import static java.lang.Math.sqrt;
import static java.lang.Math.PI;

double raiz = sqrt(25);    // sem "Math."
double area = PI * raio * raio;
```

2.3 Importando tudo de uma classe (curinga)

```
import static java.lang.Math.*;

double a = sqrt(16);
double b = pow(2, 10);
double c = max(3, 7);
```

2.4 Quando usar (e quando evitar)

- Útil em classes de utilitários matemáticos, constantes de configuração e, principalmente, em testes (JUnit: `assertEquals` , `assertTrue` , `assertThrows`).
- Evite abusar em código de produção: perder o prefixo da classe pode reduzir a legibilidade — quem lê o código pode não saber de onde veio aquele método.
- Boa prática: use `import static` para membros muito repetidos e óbvios pelo contexto (Math, constantes), não para qualquer classe.

Atenção: no DevSuperior você vai usar bastante isso na fase de testes com JUnit:

```
import static org.junit.jupiter.api.Assertions.*;
```

3. Escopo de Variáveis

3.1 Os quatro escopos em Java

Escopo	Onde é declarada	Vida útil
Variável de instância	Direto na classe (fora de métodos), sem <code>static</code>	Enquanto o objeto existir
Variável de classe (estática)	Direto na classe, com <code>static</code>	Enquanto a classe estiver carregada
Variável local	Dentro de um método ou construtor	Durante a execução do método
Variável de bloco	Dentro de <code>{ }</code> (if, for, while)	Apenas dentro daquele bloco

3.2 Exemplo integrando os quatro escopos

```
public class Contador {  
  
    static int totalInstancias = 0; // escopo de classe (compartilhado)  
    int valorAtual;                // escopo de instância (por objeto)  
  
    public Contador() {  
        totalInstancias++;  
    }  
  
    public void incrementar(int quantidade) { // 'quantidade' = escopo local  
        int novoValor = valorAtual + quantidade; // escopo local  
        if (novoValor > 100) {  
            int limite = 100; // escopo de bloco: só existe dentro do if  
            novoValor = limite;  
        }  
        // 'limite' não existe mais aqui fora do if  
        valorAtual = novoValor;  
    }  
}
```

3.3 Regra do sombreamento (shadowing)

Uma variável local pode ter o mesmo nome de uma variável de instância. Nesse caso, dentro do método, o nome se refere à variável local — a variável de instância fica "escondida", a menos que você use `this` para acessá-la explicitamente.

```
public class Produto {
    double preco;

    public void setPreco(double preco) { // parâmetro 'preco' sombreia o atributo
        this.preco = preco; // 'this.preco' = atributo | 'preco' = parâmetro
    }
}
```

Ponte com Python: é o mesmo espírito do `self` em Python: `self.preco = preco` distingue o atributo do parâmetro local.

3.4 Cuidado no for

```
for (int i = 0; i < 10; i++) {
    // i só existe aqui dentro do for
}
// System.out.println(i); // ERRO: i não existe mais fora do for
```

4. Passagem de Parâmetros: Valor x Referência

4.1 A regra central do Java

Java sempre passa parâmetros por valor. Isso vale tanto para primitivos quanto para objetos — a diferença é *o que* é copiado.

- Primitivo (int, double, boolean...): copia o valor em si. Alterar o parâmetro dentro do método NÃO afeta a variável original.
- Objeto (qualquer classe, array, String...): copia o valor da referência (o "endereço"), não o objeto. As duas variáveis (original e parâmetro) apontam para o MESMO objeto na heap.

4.2 Primitivos: nenhuma alteração é refletida

```
public static void dobrar(int x) {
    x = x * 2;
}

public static void main(String[] args) {
    int numero = 5;
    dobrar(numero);
    System.out.println(numero); // 5 – não mudou!
}
```

4.3 Objetos: alterar o CONTEÚDO reflete fora

Como a referência copiada aponta para o mesmo objeto, alterar o **estado interno** do objeto (seus atributos) é visível fora do método:

```
public static void aumentarSalario(Funcionario f) {
    f.setSalario(f.getSalario() * 1.1); // altera o objeto apontado
}

public static void main(String[] args) {
    Funcionario func = new Funcionario("Ana", 3000.0);
    aumentarSalario(func);
    System.out.println(func.getSalario()); // 3300.0 – mudou!
}
```

4.4 Mas reatribuir a referência NÃO reflete fora

Se dentro do método você fizer o parâmetro apontar para **outro objeto**, isso não afeta a variável original — porque só a cópia da referência mudou de destino:

```
public static void trocar(Funcionario f) {
    f = new Funcionario("Novo", 0.0); // f agora aponta para outro objeto
}

public static void main(String[] args) {
    Funcionario func = new Funcionario("Ana", 3000.0);
    trocar(func);
    System.out.println(func.getNome()); // "Ana" – não mudou!
}
```

Atenção: resumindo com uma frase: Java passa a REFERÊNCIA por VALOR. Você pode mudar o que o objeto contém, mas não pode fazer a variável do chamador apontar para outro objeto.

4.5 Arrays seguem a mesma regra de objetos

```
public static void zerarPrimeiro(int[] vetor) {
    vetor[0] = 0; // altera o array original (mesma referência)
}

public static void main(String[] args) {
    int[] numeros = {1, 2, 3};
    zerarPrimeiro(numeros);
    System.out.println(numeros[0]); // 0 – mudou!
}
```

Ponte com Python: é exatamente o comportamento de listas e dicts em Python: passar uma `list` para uma função e fazer `lista.append(x)` altera a lista original, mas fazer `lista = []` dentro da função cria uma lista nova e não afeta a de fora.

5. Varargs

5.1 O que resolve

Varargs permite que um método receba **uma quantidade variável de argumentos** do mesmo tipo, sem precisar sobrecarregar o método várias vezes ou obrigar o chamador a montar um array manualmente.

5.2 Sintaxe

```
public static int somar(int... numeros) {
    int total = 0;
    for (int n : numeros) {
        total += n;
    }
    return total;
}

// Chamadas possíveis:
somar();           // 0 argumentos -> total = 0
somar(5);          // 1 argumento
somar(1, 2, 3, 4, 5); // 5 argumentos
```

5.3 Por baixo dos panos: é um array

Dentro do método, `numeros` é tratado como um `int[]` normal. Você também pode passar um array diretamente:

```
int[] valores = {10, 20, 30};
int resultado = somar(valores); // funciona, equivale a somar(10, 20, 30)
```

5.4 Regras importantes

- Só pode haver UM parâmetro varargs por método.
- O varargs deve ser o ÚLTIMO parâmetro da lista.
- Pode conviver com outros parâmetros normais antes dele.

```
// Válido: parâmetro fixo antes do varargs
public static void logar(String nivel, String... mensagens) {
    for (String m : mensagens) {
        System.out.println "[" + nivel + " ] " + m);
    }
}

logar("INFO", "Iniciando sistema", "Conectando ao banco");

// INVÁLIDO – varargs não pode vir antes de outro parâmetro
// public static void logar(String... mensagens, String nivel) { }
```

Ponte com Python: varargs é o equivalente Java ao `*args` do Python — mesma ideia, mesma restrição de só poder ter um por assinatura e vir por último.

6. Classes Aninadas: Internas, Locais e Anônimas

6.1 Panorama geral

Tipo	Onde é declarada	Acessa membros da classe externa?
Estática aninada (static nested)	Dentro da classe, com <code>static</code>	Só membros estáticos
Interna (inner class)	Dentro da classe, sem <code>static</code>	Sim, inclusive não-estáticos
Local	Dentro de um método	Sim, e variáveis locais "efetivamente finais"
Anônima	Na hora de instanciar, sem nome	Sim, mesma regra da local

6.2 Classe estática aninada (static nested class)

Não precisa de uma instância da classe externa para existir. É basicamente uma classe normal que só "mora dentro" de outra por organização (ex.: uma classe auxiliar usada só por aquela classe).


```
public class Agenda {

    static class Estatisticas { // classe estática aninada
        int totalContatos;
        Estatisticas(int total) {
            this.totalContatos = total;
        }
    }

    public Estatisticas gerarEstatisticas() {
        return new Estatisticas(10);
    }
}

// Uso externo:
Agenda.Estatisticas stats = new Agenda.Estatisticas(5);
```

6.3 Classe interna (inner class)

Está ligada a uma instância específica da classe externa. Para criar uma, você precisa primeiro ter um objeto da classe de fora.

```
public class Pedido {
    double valorTotal;

    class ItemPedido { // classe interna (sem static)
        String descricao;
        double preco;

        void aplicarDesconto() {
            // acessa o atributo da classe externa diretamente
            valorTotal -= preco * 0.1;
        }
    }
}

// Uso externo – precisa de uma instância de Pedido primeiro:
Pedido pedido = new Pedido();
Pedido.ItemPedido item = pedido.new ItemPedido();
```

Atenção: classes internas não-estáticas são menos comuns no dia a dia; aparecem bastante em implementações de estruturas de dados (ex.: um `Node` dentro de uma `LinkedList`) e em listeners de interface gráfica.

6.4 Classe local

Declarada dentro do CORPO de um método. Só existe e só pode ser usada dentro daquele método.

```

public void processarPedido() {

    class Validador { // classe local – só existe dentro deste método
        boolean valido(double valor) {
            return valor > 0;
        }
    }

    Validador v = new Validador();
    System.out.println(v.valido(100.0)); // true
}
// Validador não existe fora de processarPedido()

```

6.5 Classe anônima

Uma classe sem nome, declarada e instanciada ao mesmo tempo — normalmente usada para implementar uma interface ou estender uma classe "na hora", quando você só vai usar aquela implementação uma única vez.

```

interface Operacao {
    int aplicar(int a, int b);
}

public class Calculadora {
    public static void main(String[] args) {

        Operacao soma = new Operacao() { // classe anônima
            @Override
            public int aplicar(int a, int b) {
                return a + b;
            }
        };

        System.out.println(soma.aplicar(3, 4)); // 7
    }
}

```

Antes das *lambdas* (Java 8+), classes anônimas eram o único jeito de passar "comportamento" como argumento. Hoje, para interfaces funcionais (com um único método abstrato), o mesmo código fica assim:

```

Operacao soma = (a, b) -> a + b; // lambda substitui a classe anônima
System.out.println(soma.aplicar(3, 4)); // 7

```

Ponte com Python: classe anônima + interface funcional é o parente distante das funções lambda e closures do Python — a diferença é que em Java, historicamente, precisava desse "embrulho" de classe para simular comportamento passado como dado.

7. Annotations

7.1 O que são

Annotations (anotações) são metadados que você anexa a classes, métodos, atributos ou parâmetros. Elas não mudam o comportamento do código diretamente — servem como instruções para o compilador, para ferramentas, ou para frameworks (Spring, JUnit, JPA) lerem em tempo de execução via *reflection*.

7.2 Annotations nativas mais usadas

Annotation	Para que serve
<code>@Override</code>	Avisa o compilador que o método sobrescreve um método da superclasse/interface. Gera erro de compilação se não sobrescrever nada de fato.
<code>@Deprecated</code>	Marca um elemento como obsoleto — gera aviso de compilação para quem usar.
<code>@SuppressWarnings</code>	Suprime avisos específicos do compilador (ex.: <code>unchecked</code>) em um trecho de código.
<code>@FunctionalInterface</code>	Marca uma interface como tendo exatamente um método abstrato (habilitando uso com lambda).

```

public class Animal {
    public void emitirSom() {
        System.out.println("Som genérico");
    }
}

public class Cachorro extends Animal {

    @Override
    public void emitirSom() {    // o compilador CONFIRMA que isso sobrescreve Animal.emitirSom()
        System.out.println("Au au");
    }

    @Deprecated
    public void metodoAntigo() {
        // ainda funciona, mas avisa quem chamar que deve migrar
    }

    @SuppressWarnings("unchecked")
    public void metodoComWarningSuprimido() {
        List lista = new ArrayList(); // sem generics -> gera warning, aqui suprimido
    }
}

```

7.3 Onde você já usa annotations sem perceber

- **JUnit:** `@Test` , `@BeforeEach` , `@AfterEach` — marcam métodos que o framework deve executar automaticamente.
- **Spring** (próximo passo do seu roadmap): `@RestController` , `@Autowired` , `@Entity` — dizem ao framework como instanciar e conectar suas classes.
- **Lombok** (comum no mercado): `@Getter` , `@Setter` , `@Data` — geram código repetitivo automaticamente.

7.4 Criando uma annotation própria (visão inicial)

Você não vai usar isso ainda na fase de POO básica, mas é bom reconhecer a sintaxe — ela vai aparecer bastante quando chegar em Spring Boot:

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME) // disponível em tempo de execução
@Target(ElementType.METHOD)       // só pode ser usada em métodos
public @interface TesteUnitario {
    String autor() default "desconhecido";
}

// Uso:
public class MinhaClasseDeTeste {

    @TesteUnitario(autor = "Jader")
    public void testarSoma() {
        // ...
    }
}

```

Atenção: não se preocupe em dominar a criação de annotations agora — o objetivo aqui é apenas reconhecer a sintaxe `@interface` e entender que frameworks usam reflection para ler essas anotações em tempo de execução. Isso volta com força total quando você chegar em Spring Boot no seu roadmap.